

ITSC-3146

Test 1 Review

Feel free to ask questions at any time!

Agenda

Covering everything in Modules 1-3

1. Computer Organization (Briefly)
2. Command Line
3. Operating System Basics
4. C++
 - a. Structs
 - b. Pointers
5. Processes
 - a. Concepts
 - b. Implementation in C++
6. Questions

Computer Organization - Memory (1/2)

- Main memory is a collection of supercells
 - Typically implemented with RAM
 - Comprised of smaller 1 bit cells
 - Stores data (which we access and point to with pointers!)
- Cache is like a smaller version of the main memory
 - Stores commonly used values for faster retrieval in the future
- Memory, at the lowest level, is comprised of registers
 - Registers for:
 - The program counter
 - What line the program currently is at
 - Stack pointer
 - Address at the top of stack
- All of this helps us store variables quickly and reliably!

0x00	01101000
0x01	10101001
0x02	10100000
0x03	00000000
0x04	10001010
0x05	01010101
0x06	10010101
0x08	01100110
0x08	10010110
0x09	01101001

Computer Organization - Input / Output (I/O) (2/2)

- Most programs need to take input of some kind
- Multiple ways of actually implementing this system
 - Polling (think back to the donkey example in the slides!)
 - Continually asking if the I/O device has anything new to send
 - Costly and inefficient usually
 - Interrupts
 - Computer sends a signal to the device letting it know it needs I/O
 - The I/O device then interrupts the program when it has completed its task

- ◆ CPU *stops* current activity
- ◆ CPU *asks* if device is ready, e.g.
 - ◆ Device has new input
 - ◆ Device is done with previous command/output
- ◆ If yes, CPU services device
- ◆ CPU moves to next device
- ◆ Once done, CPU *resumes* previously stopped activity
- ◆ CPU repeats above steps *periodically*

Sequence of activities...

- ◆ CPU executing instructions
- ◆ *Interrupt* received
- ◆ CPU
 - ◆ Finishes current instruction
 - ◆ Recognizes interrupt
 - ◆ Saves current state
 - ◆ Services interrupt
 - ◆ Resumes normal activity

The Command Line - File Management (1/2)

- When you are using the command line in this class, you are using BASH
 - Known as a shell which gives you a textual way of interacting with the system
 - Relatively easy to understand once you can visualize what is happening
- Basic commands to know
 - cd
 - Stands for “change directory” and allows you to change what folder (directory) you are currently in
 - ls
 - Displays all files and directories inside of you current directory
 - mkdir
 - Makes a directory where the current path is located
- For more commands, check out [this website](#) or [this one](#).

The Command Line - C++ Compiler (2/2)

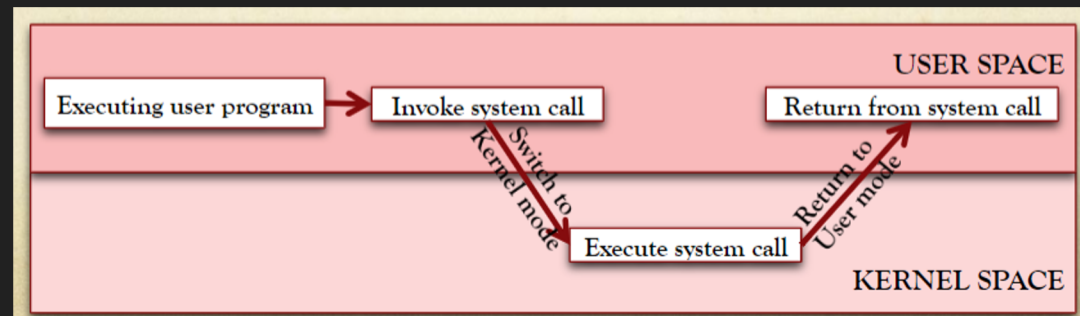
- Just like in other compiled languages (like Java) need to compile code before we can run it
 - To do this we use the g++ command
 - `g++ <NAME OF FILE HERE> -o <NAME OF EXECUTABLE HERE>`
 - To execute the program, just need to tell bash to run it for us
 - `./<NAME OF EXECUTABLE HERE>`
- Using this, along with the file management commands, should be able to compile and run code easily

Operating System Basics (1/4)

- An operating system is something that sits between hardware and software
 - This is the first half of what we are learning in this class
 - Gives access to hardware components through software
 - Program execution
 - Memory management
 - File management
 - I/O Management
 - ... And more!
- OS design separates how (mechanism) from what/when/which (policy)
- Two different system operation modes
 - User
 - Executes programs on behalf of the user in a protected mode
 - No direct access to hardware with restricted access to instructions and memory areas
 - Kernel
 - Executes on behalf of operating system in a privileged mode
 - Complete access to all areas of the hardware, memory, and instruction sets

Operating System Basics (2/4)

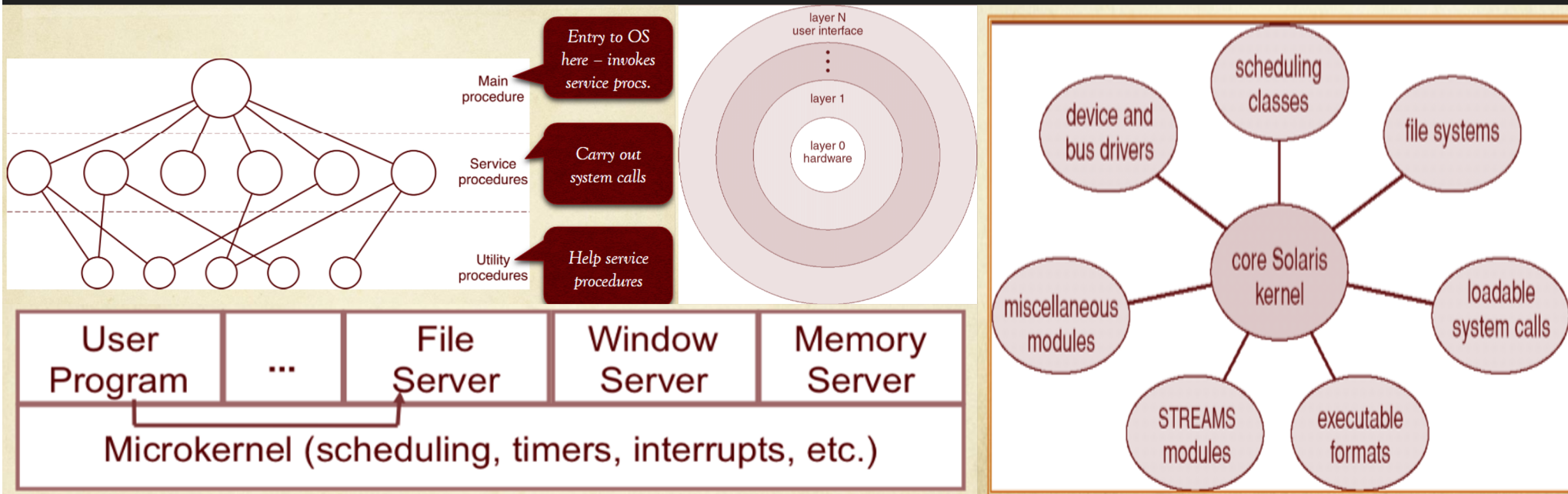
- Some programs need access to the things available in Kernel mode but don't have that form of access
 - This is achieved through a System call
 - Interfaces a running program with the OS
 - Provides it controlled entry into kernel mode
 - Gives the program certain parameters and boundaries
- A system call causes the system to switch to Kernel mode



Operating System Basics (3/4)

- Several ways to architect an operating system
 - Monolithic
 - Entire OS within a single program
 - All programs operating in Kernel mode
 - Can get very messy - any part can bring any other part down
 - Layered
 - Each part of the OS gets layers that are independent of each other
 - Microkernel
 - Split all functionality into smaller modules
 - Core is known as microkernel and runs in kernel mode
 - All other modules interface with the microkernel for kernel mode
 - Modern operating systems use a mix of all in a hybrid approach

Operating System Basics - OS Diagrams (4/4)



C++ Basics - Pointers (1/2)

- Need a way to access the direct memory location of a variable
 - This is done through pointers!
 - Take this code for example:

```
int test = 5;  
int *testPtr = &test;  
cout << *testPtr;
```

- This will print out 5!
 - &test gives us the memory address
 - *testPtr retrieves the value stored at a memory address
- Declare a pointer variable with the * operator
 - Like so:

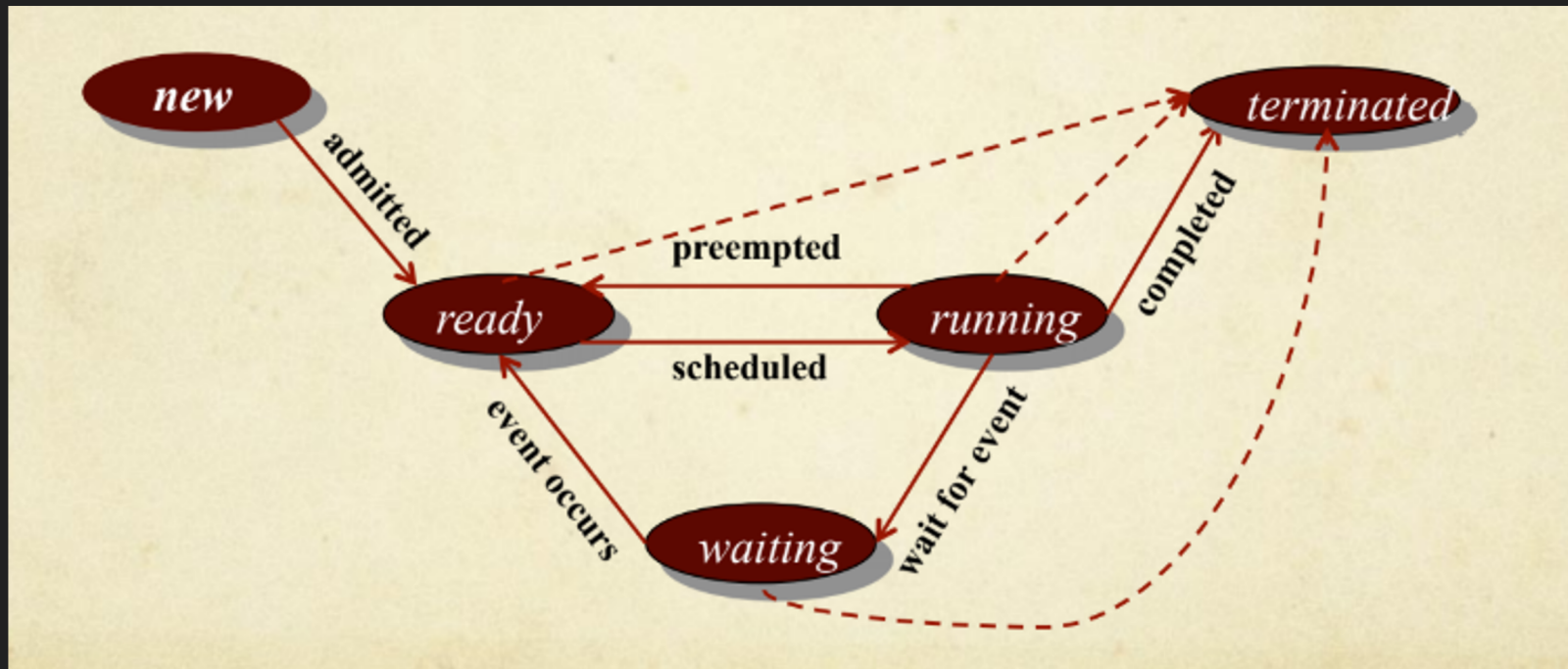
```
int *testPtr;
```

C++ Structs (2/2)

- A **struct** is essentially a user defined custom data structure. With a struct, you can store multiple fields together.
- It is similar to a **class**, but lightweight
- To access struct members, use the `.` operator
- To access struct members when the struct is a pointer, use the `->` operator.

Processes - Intro (1/6)

- A process is a running instance of the program
- Processes are created from cloning



Processes - Process Control Block (2/6)

Process control block used for this

- ◆ Process ID
- ◆ Process State (ready, running etc.)
- ◆ Program Counter – address of next instruction to be executed
- ◆ Registers – general purpose registers, stack pointer etc.
- ◆ Scheduling information
- ◆ Memory management information
- ◆ Accounting information – time limits, etc.
- ◆ ...

Process ID
Process State
Program Counter
Registers
Memory limits
....

Processes - Creating Processes (3/6)

- In Unix based systems, processes are cloned from an existing process (init)
- With C++ we can call `fork()` to create a process
- The `fork()` call returns a different value

```
pid_t id = fork();  
if(id == -1) {  
    std::cout << "Error creating process\n";  
} else if (id == 0) {  
    std::cout << "I'm a child process!\n";  
} else {  
    std::cout << "I just became a parent!\n";  
}
```

Processes - execvp (4/6)

- The execvp command can be used to change a process' memory image
- Copy on write - start with shared memory and only make a copy when data is modified

```
pid_t id = fork();
if(id == -1) {
    std::cout << "Error creating process\n";
} else if (id == 0) {
    // child process functionality
    char* args[] = {"echo", "hello", NULL};
    execvp(args[0], args);
} else {
    std::cout << "I just became a parent!\n";
}
```

Processes - Orphans and Zombies (5/6)

- OS maintains a hierarchy
- If a parent terminates before the child, the child becomes an orphan and is adopted by the init process
- If a child is terminated before the parent, the child becomes a zombie and can be reaped by its parent
 - The OS provides the wait command for this

Processes - Typical Steps (6/6)

[*hw*] Save program counters & some registers on stack

[*hw*] Load program counter specified in *interrupt vector*

◆ Interrupt vector → has address of *interrupt service routine*

[*asm*] Save context info (registers, etc.) in *PCB* of curr process

[*asm*] Set up new stack

[*C*] Remaining work for specific interrupt type

[*C-sched*] Choose new process to be scheduled

[*asm*] Restore context info for new process & start it

Code Review

Code Review- Data Types

Given the following code, what is the output?

Answer: 2

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5
6      float num1 = 1.45;
7      double num2 = 2;
8      int result;
9
10     result = num1 * num2;
11
12     cout << result << endl;
13
14     return 0;
15 }
```


Code Review- Pointers

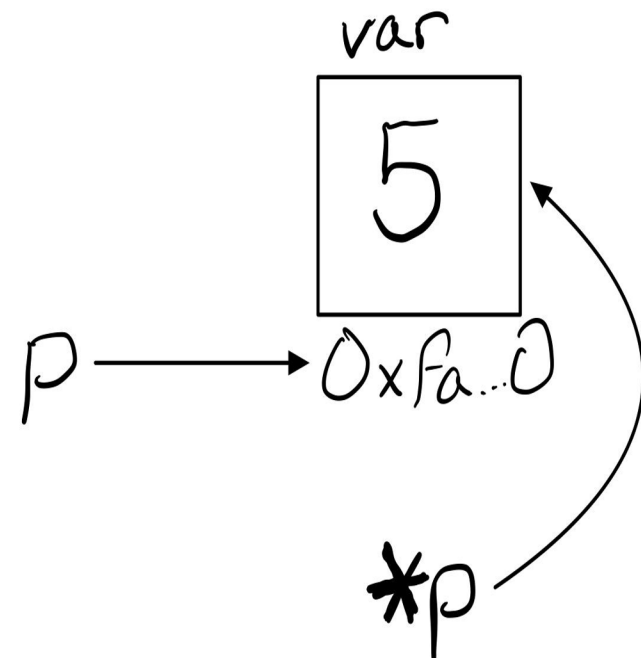
- 1) print var → ?
- 2) print &var → ?
- 3) print p → ?
- 4) print *p → ?
- 5) print &p → ?

Pointer Basics

```
int var = 5;  
int* p;  
p = &var
```

-OR-

```
int var = 5;  
int* p = &var;
```



Code Review- Pointers

```
//To Declare a Pointer:  
int myInt = 15;  
int *myPointer = &myInt;  
  
cout << myPointer << endl; //value of myPointer  
cout << &myInt << endl; //memory address of myInt  
  
cout << *myPointer << endl << endl; //value pointed to by myPointer  
cout << myInt << endl; //value stored in myInt
```

Code Review- Structs and Functions

```
struct point{
    float x;
    float y;
};

void displayArray(point myPoints[]){
    for(int i=0; i<2; i++){
        cout << myPoints[i].x << " " << myPoints[i].y << endl;
    }
}

int main()
{
    struct point point_list[2] = {
        {1.0,2.5},
        {3.2,5.4}
    };

    displayArray(point_list);

    return 0;
}
```

Code Review- Structs and Functions

Struct called point containing two fields of type float x and y

```
struct point{  
    float x;  
    float y;  
};
```

```
void displayArray(point myPoints[]){  
    for(int i=0; i<2; i++){  
        cout << myPoints[i].x << " " << myPoints[i].y << endl;  
    }  
}
```

return
type

Takes in an point
array as a parameter
in the displayArray
function

accessing index 0's x and
y fields

```
int main()  
{
```

```
    struct point point_list[2] = {  
        {1.0,2.5},  
        {3.2,5.4}  
    };
```

declaring an array
of points each index
contains an x and
y

```
    displayArray(point_list);
```

calling displayArray and
passing point_list as
an argument

```
    return 0;
```

```
}
```

Coding Review - Forks

Assume that the code given is executed once.

1. Where is the first process created?
2. How many times (if any) will "Hi" be printed to the console?

```
main()  
{  
  
    for(i=0; i < 2 ; i++)  
    {  
        fork();  
        print ("Hi");  
    }  
}
```

Coding Review - Forks

Assume that the code given is executed once.

1. Where is the first process created? Main Process
2. How many times (if any) will "Hi" be printed to the console? 6 times

```
main()  
{
```

```
  for(i=0; i < 2; i++)  
  {
```

```
    fork();  
    print("Hi");
```

```
  }
```

```
}
```

